

سيجما ماركت - Sigma Market

دليل الشرح التفصيلي للأكواد الهندسية (Code Explanation Guide)

التاريخ: 30 سبتمبر 2026

الموديول: Customer Authentication

معرف الالتزام (Commit): cf200fd

اسم الفرع (Branch): feature/customer-account

1. أكواد الـ Backend (خلفية النظام - Node.js / Express / TypeScript)

أ. ملف التحكم والـ Business Logic: `src/modules/users/users.controller.ts`

يتضمن هذا الملف دوال التحقق وتعديل كلمة المرور للمستخدمين بعد التأكد من الهوية وحماية التشفير:

```
// 1. Zod مخطط التحقق من المدخلات باستخدام مكتبة
export const changePasswordSchema = z.object({
  body: z.object({
    currentPassword: z.string().min(1, 'Current password is required'),
    newPassword: z.string().min(8, 'New password must be at least 8 characters long')
  })
});

// 2. دالة تغيير كلمة المرور والتحقق من التشفير
export const changeUserPassword = async (req: Request, res: Response) => {
  try {
    const { id } = req.params;
    const { currentPassword, newPassword } = req.body;

    // Prisma ORM البحث عن المستخدم في قاعدة البيانات عبر
    const user = await prisma.user.findUnique({ where: { id } });
    if (!user) {
      return sendError(res, 'User not found.', 404);
    }

    // bcryptjs المخزن باستخدام Hash مقارنة كلمة المرور الحالية بالـ
    const isValid = await comparePassword(currentPassword, user.passwordHash);
    if (!isValid) {
      return sendError(res, 'Current password is incorrect.', 400);
    }

    // تشفير كلمة المرور الجديدة وتحديث قاعدة البيانات
    const newHash = await hashPassword(newPassword);
    await prisma.user.update({
      where: { id },
      data: { passwordHash: newHash }
    });

    return sendSuccess(res, 'Password changed successfully.');
```

```
  } catch (error: any) {
    return sendError(res, 'Failed to change password.', 500, error.message);
  }
};
```

الشرح التفصيلي للـ Backend Controller:

- **changePasswordSchema**: يقوم بفحص صيغة المدخلات قبل وصولها للبيانات، ويشترط ألا تقل كلمة المرور الجديدة عن 8 أحرف.
- **comparePassword**: تفحص كلمة المرور الحالية التي أدخلها المستخدم وتضمن مطابقتها لـ passwordHash المخزن في جدول users.
- **hashPassword**: تشفير كلمة المرور الجديدة باستخدام bcryptjs بـ 10 Salt Rounds لحمايتها من الاختراق قبل التخزين.

ب. ملف المسارات والموجهات: `src/modules/users/users.routes.ts`

```
// مسار حماية وتغيير كلمة المرور للمستخدم
router.put(
 ('/:id/password',
    authenticate,
    requireOwnership('id'),
    validate(changePasswordSchema),
    changeUserPassword
  );
```

الشرح التفصيلي للـ `Routes Middleware`:

- `authenticate`: يتأكد أن المستخدم مسجل ومعه رمز JWT صالح وموثوق.
- `requireOwnership('id')`: جدار أمني يتأكد أن العميل يحاول تعديل كلمة المرور ****لحسابه الشخصي فقط**** ويمنع التلاعب بأرقام حسابات الآخرين (Forbidden 403).

2. أكواد إدارة الجلسات والمصادقة بالواجهة الأمامية: public/js/auth.js

```
const Auth = {
  token: localStorage.getItem('swp_token') || null,
  user: JSON.parse(localStorage.getItem('swp_user') || 'null'),

  // حفظ الجلسة بالذاكرة وتحديث الترويسة
  setSession(token, user) {
    this.token = token;
    this.user = user;
    localStorage.setItem('swp_token', token);
    localStorage.setItem('swp_user', JSON.stringify(user));
    this.updateUI();
  },

  // مسح بيانات الجلسة عند الخروج
  clearSession() {
    this.token = null;
    this.user = null;
    localStorage.removeItem('swp_token');
    localStorage.removeItem('swp_user');
    this.updateUI();
  },

  // Bearer Token مع الـ Authorization Header تجهيز
  getHeaders() {
    const headers = { 'Content-Type': 'application/json' };
    if (this.token) {
      headers['Authorization'] = `Bearer ${this.token}`;
    }
    return headers;
  },

  // المعالجة الآمنة لانتهااء الجلسة (401 Unauthorized)
  handleApiUnauthorized() {
    this.clearSession();
    const modal = document.getElementById('auth-modal');
    if (modal) modal.classList.add('active');
  }
};
```

الشرح التفصيلي لـ Auth State Manager:

- **setSession**: تحفظ الـ JWT وبيانات العميل بـ `localStorage` لتظل الجلسة نشطة عند عمل `Refresh` للصفحة.
- **getHeaders**: تضمن إرسال `<token Bearer <Authorization:` تلقائياً مع أية طلبات موجهة للـ API المحمية.
- **handleApiUnauthorized**: عند انتهاء صلاحية الـ Token في السيرفر، تسمح الجلسة وتفتح نافذة تسجيل الدخول فوراً لحماية العميل.

3. أكواد التفاعل وإدارة النماذج والـ APIs بالواجهة: `public/js/app.js`

أ. معالجة التسجيل والدخول والتحقق من البيانات:

```
authForm.addEventListener('submit', async (e) => {
  e.preventDefault();
  resetAuthAlert();

  const email = document.getElementById('auth-email').value.trim();
  const password = document.getElementById('auth-password').value;

  // Regex فحص صيغة البريد الإلكتروني عبر
  const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
  if (!email || !emailRegex.test(email)) {
    return showAuthError('يرجى إدخال بريد إلكتروني صحيح');
  }

  // فحص تطابق كلمة المرور والطول عند التسجيل
  if (isRegistering) {
    const fullName = document.getElementById('auth-fullname').value.trim();
    const confirmPassword = document.getElementById('auth-confirm-password').value;

    if (!fullName || fullName.length < 2) return showAuthError('الاسم الكامل يجب ألا يقل عن حرفين');
    if (password.length < 8) return showAuthError('كلمة المرور يجب ألا تقل عن 8 أحرف');
    if (password !== confirmPassword) return showAuthError('كلمتا المرور غير متطابقتين');
  }

  // Network Response تعطيل الزر لمنع الضغط المزدوج أثناء انتظار الـ
  submitBtn.disabled = true;
  submitBtn.textContent = 'جارى المعالجة...';

  try {
    const endpoint = isRegistering ? '/api/auth/register' : '/api/auth/login';
    const res = await fetch(endpoint, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(payload)
    });
    const data = await res.json();

    if (data.success) {
      Auth.setSession(data.data.token, data.data.user);
      modal.classList.remove('active');
    } else {
      showAuthError(data.message || 'بيانات المصادقة غير صحيحة');
    }
  } finally {
    submitBtn.disabled = false;
  }
});
```

ب. إدارة عناوين التوصليل (Addresses CRUD Logic):

```
// استعلام وجلب كافة عناوين المستخدم
async function loadUserAddresses() {
  const res = await fetch('/api/users/addresses', { headers: Auth.getHeaders() });
  const data = await res.json();

  if (data.success && data.data.addresses) {
    container.innerHTML = data.data.addresses.map(addr => `

      ${escapeHtml(addr.street)}

      ${addr.isDefault ? 'العنوان الافتراضي' : ''}
      

تعيين كافتراضي
        حذف



    `).join('');
  }
}

// تعيين العنوان كعنوان افتراضي
window.setDefaultAddressAction = async function(id) {
  const res = await fetch(`/api/users/addresses/${id}/default`, {
    method: 'PATCH',
    headers: Auth.getHeaders()
  });
  const data = await res.json();
  if (data.success) loadUserAddresses();
};
```

4. أكواد الاختبار الآلي بنظام Vitest: tests/auth.test.ts

```
it('Should allow authenticated user to change password and login with new password', async () => {
  // جلب بيانات المستخدم الموثق 1.
  const meRes = await request(app)
    .get('/api/auth/me')
    .set('Authorization', `Bearer ${authToken}`);

  const userId = meRes.body.data.user.id;

  // طلب تغيير كلمة المرور 2.
  const res = await request(app)
    .put(`/api/users/${userId}/password`)
    .set('Authorization', `Bearer ${authToken}`)
    .send({ currentPassword: testUser.password, newPassword: 'NewPassword123!' });

  expect(res.status).toBe(200);
  expect(res.body.success).toBe(true);

  // التحقق من إمكانية الدخول بكلمة المرور الجديدة 3.
  const loginRes = await request(app)
    .post('/api/auth/login')
    .send({ email: testUser.email, password: 'NewPassword123!' });

  expect(loginRes.status).toBe(200);
  expect(loginRes.body.data.token).toBeDefined();
});
```